

Ventor-QTest: Threat-Model-Driven Verification of Vendor-Hosted LLM APIs

Tencent Zhuque Lab

Xiangfan Wu, Zonghao Ying, Huiyu Wu, Xing Zheng,
Huangsheng Cheng, Xiaorong Shi, Jing Guo

Abstract

As large language models become increasingly widespread, third-party providers that deploy open-weight models have become an important part of the ecosystem. Auditing the quality of their inference APIs is therefore an open problem. We formalize hosted model routing as a stochastic process and propose **Ventor-QTest**, a composite black-box audit that requires no probability information from the target API. Its repeated-request component sends each frozen constrained context to the target multiple times, reconstructs a categorical output distribution from the returned text counts, and reports *average fidelity loss* (AFL) as a null-bias-corrected, within-window mean coarsened-KL statistic. Its long-sequence component uses independent runs to report *extreme fidelity loss* (EFL) through the empirical upper tail of a run-level reference-centered-surprisal statistic. Across three logprob-capable route conditions, AFL shows strong linear descriptive agreement with a logprob-derived coarsened-KL comparator. Across seven route snapshots, 20-run sequence probes reveal route-specific EFL variation. AFL and EFL have little detectable route-level association with GPQA-Diamond accuracy. In contrast, pronounced EFL coincides with a decline in Terminal-Bench pass rate as task exposure increases. This pattern may arise because correctness in long-horizon tasks is more sensitive to extreme fidelity loss. These results motivate reporting AFL and EFL jointly, particularly when auditing long-horizon agentic tasks. The open-source implementation is available at https://github.com/Tencent/AI-Infra-Guard/tree/main/services/api_checker/ventor_qtest.

1. Introduction

Frontier large language models (LLMs) are reshaping software development, information retrieval, and data analysis, and they increasingly serve as core reasoning components in real-world workflows. Agentic coding systems make this shift concrete: Codex and Claude Code can inspect codebases, edit files, invoke development tools, run tests, and execute multi-step tasks [1, 37]. As these systems move beyond text generation to act on external state, the behavior of the model serving each request becomes a systems-level reliability concern.

Open-weight releases have simultaneously become a major part of the LLM ecosystem. The number, adoption, and downstream reuse of public models on platforms such as Hugging Face continue to grow, sustaining an active ecosystem of base models, quantized variants, adapters, and application-specific derivatives [23, 29]. Open weights, however, do not imply inexpensive self-hosting. For example, the official Kimi K2 deployment guide specifies that the smallest deployment unit for its FP8 weights at a 128K context length on H200 or H20 hardware comprises 16 GPUs [35]. Such infrastructure requirements encourage developers and organizations to access open models through third-party inference APIs, moving execution outside the model publisher’s trusted environment.

This shift creates a verification gap. Model names in hosted LLM APIs are claims rather than cryptographic attestations in the sense of an evidence-producing remote-attestation architec-

ture [3]. A client who requests a particular checkpoint normally observes only a response string and provider-controlled metadata exposed by the serving APIs [10, 38, 44]. The serving path may instead use an older checkpoint, a cheaper substitute, a quantized deployment, or a different decoding stack. Quantization can affect inference efficiency and task accuracy [24, 48, 54]; separately, deployment-verification studies show that serving configurations can alter observable output behavior [5, 17, 42, 58]. Some changes are benign engineering choices, while others may violate an advertised contract, and black-box observations alone do not establish which explanation is true. We therefore ask: *Can black-box API queries measure the behavioral deviation of a third-party deployment from a trusted reference?* This paper studies this question as *deployment verification*: testing whether a hosted endpoint behaves like a trusted reference deployment under a stated configuration.

We instantiate this view in **Ventor-QTest**, a composite black-box audit requiring probabilities only from a trusted reference. The repeated-request component sends each frozen constrained context to the target M times, maps the returned texts into predeclared categories, reconstructs the corresponding target output distribution, and compares it with the trusted reference. Its output S_r is the *average fidelity loss* (AFL): a null-bias-corrected, within-window mean coarsened-KL statistic. Independent long-sequence probes retain the empirical distribution of all observed run-level centered-surprisal deviations; their empirical upper-tail behavior is the *extreme fidelity loss* (EFL). AFL and EFL have different units and are reported together, not collapsed by a manually calibrated threshold or weighted sum.

Experiments across hosted DeepSeek route snapshots show descriptive agreement between AFL and a logprob-derived comparator, and reveal route-specific EFL. AFL and EFL have little detectable route-level association with GPQA-Diamond accuracy [40]. In contrast, pronounced EFL coincides with a decline in Terminal-Bench pass rate as task exposure increases [32]. This pattern may arise because correctness in long-horizon tasks is more sensitive to extreme fidelity loss.

This paper makes four contributions:

- We formalize vendor-hosted inference as a route-level stochastic process and distinguish persistent, intermittent, substitution, and adaptive-routing deviations.
- We develop a text-only repeated-request estimator of AFL for a predeclared finite outcome map and demonstrate strong linear descriptive agreement with a logprob-derived coarsened-KL comparator on three logprob-capable route conditions.
- We introduce a complementary long-sequence probe of EFL and report the empirical distribution of all observed run-level centered-surprisal deviations, including upper-tail summaries.
- We evaluate AFL and EFL across hosted route snapshots and contrast their downstream relationships on GPQA-Diamond and Terminal-Bench.

2. Scope, Threat Model, and Assumptions

2.1. Verification Target

The object under test is a *served model instance*, not weights in isolation. We represent an advertised claim as

$$C = (m, v, q, \phi, \mathcal{E}), \quad (1)$$

where m is the model family, v is the checkpoint or version, q is the numerical precision or quantization, ϕ denotes decoding and serving parameters, and $\mathcal{E}$ denotes API semantics such as

tool-call formatting. A test can reject behavioral consistency with C on a probe distribution. It cannot, from text alone, prove which component changed or whether the change was intentional.

The claim in Equation (1) describes the full advertised service, but the present Ventr-QTest instantiation tests only its text-generation projection. It does not validate the full API semantics $\mathcal{E}$, including tool execution, multimodal preprocessing, or agent-environment interaction.

The verifier controls a client and has access to a trusted reference endpoint P for the claimed instance. A static target would expose one endpoint distribution Q_r , but a deployed route can switch replicas, numerical formats, inference kernels, or routing policies across serving windows. We therefore model route r as a stochastic process $\{Q_{r,b}\}_{b \in \mathcal{B}}$, where b indexes an independently sampled audit window. Given challenges $x \sim \mathcal{G}$, full stochastic consistency is

$$H_0 : Q_{r,b}(\cdot \mid x, \phi) = P(\cdot \mid x, \phi) \quad \text{for all } x \in \mathcal{G} \text{ and } b \in \mathcal{B}. \quad (2)$$

Equation (2) is the ideal global consistency condition; its violation defines a served-distribution deviation, not a statement that the target is worse on downstream tasks. The finite audit below does not test this universal condition directly.

This process view separates persistent and intermittent latent divergence. Let T_x be the declared outcome coarsening for challenge x and define the latent window divergence

$$L_{r,b} = \mathbb{E}_{x \sim \mathcal{G}} D_{\text{KL}}(T_x Q_{r,b} \parallel T_x P), \quad A_r = \mathbb{E}_b[L_{r,b}]. \quad (3)$$

The cross-window mean A_r describes a persistent shift. For a task exposed to m independently sampled serving windows, define the latent excess-maximum functional

$$X_r(m) = \mathbb{E} \left[\max_{1 \leq k \leq m} L_{r,k} \right] - A_r. \quad (4)$$

Unlike a thresholded degraded/not-degraded label, $X_r(m)$ retains the magnitude of the latent upper tail and is nondecreasing in the number of sampled windows m . A route can therefore have a modest A_r but a material $X_r(m)$ when severe deviations occur intermittently. This makes explicit a stochastic serving-window dimension that is implicit in route-level capability verification such as KVV [34, 36]. The primary experiment does not directly estimate either A_r or $X_r(m)$; they motivate the two observable audit outputs summarized in Table 1.

Table 1 | Latent objects and observed Ventr-QTest statistics. AFL and EFL name the two reported observables; they are not interchangeable estimators of a common loss.

Object	Meaning	Directly estimated here?
A_r	Cross-window mean latent coarsened KL	No; most routes have one repeated-request window
$S_{r,b}$	AFL: within-window, across-context bias-corrected coarsened-KL statistic	Yes
$X_r(m)$	Expected excess maximum of latent window KL	No
$D_{r,b}, \hat{F}_r^D$	EFL: run-level centered-surprisal statistic and its observed empirical upper tail	Yes; neither is KL

For tasks whose desired behavior is defined by the trusted reference, greater model-output exposure may create more opportunities to encounter an altered serving window. We treat this statement as motivation for exploratory downstream analysis rather than a deterministic task-success model. Deviations may hurt, leave unchanged, or occasionally improve a task outcome, while timeouts, rate limits, and tool semantics provide additional non-model pathways to failure.

2.2. Provider Behavior Classes

We consider five increasingly adversarial cases.

Operational deviation. An honest vendor uses the claimed weights but differs in inference engine, parameter validation, tokenization, batching, or numerical kernels.

Undisclosed quantization. The vendor serves a lower-precision instance, such as FP8 or FP4, without making the distinction clear to the client. FP8 is an established low-precision format, whereas FP4 is an actively studied lower-precision regime [33, 48]; deployment verification studies show that quantization can change observable model behavior without implying malicious intent [17, 42, 58].

Intermittent fidelity degradation. A heterogeneous replica pool or time-varying router usually serves a close configuration but occasionally selects a substantially altered one. This case may leave a moderate average deviation while enlarging the upper tail across independent runs.

Substitution or metadata forgery. The vendor serves another checkpoint or model and may return the requested model name in provider-controlled metadata. We therefore treat response metadata as evidence to preserve, not as ground truth.

Adaptive selective routing. The vendor observes the request, classifies whether it is an audit, and routes likely probes to a faithful model while routing other traffic to a cheaper or altered model. It may maintain state across requests, cache known answers, or fine-tune a substitute to imitate public probes. Recent work studies adversarial substitution and diluted or mixed routing [5, 16, 56] and demonstrates that finite-budget fingerprints can be spoofed by a model trained to mimic their probe behavior [55].

The adaptive case exposes the limitation of a fixed question set. Let $h(x) \in \{0, 1\}$ be the provider’s probe classifier and let R be a substitute. Selective routing implements

$$Q_{\text{route}}(\cdot \mid x) = \begin{cases} P(\cdot \mid x), & h(x) = 1, \\ R(\cdot \mid x), & h(x) = 0. \end{cases} \quad (5)$$

For a public finite set $\mathcal{S}$, the dictionary classifier $h(x) = \mathbb{1}[x \in \mathcal{S}]$ defeats any audit restricted to $\mathcal{S}$. A hidden set delays this attack but does not remove it: leakage, repeated observation, or surrogate reconstruction can turn the hidden set into a known one. Online selective routing can therefore defeat audits based on recognizable finite probes [5, 16, 39, 55, 56]. Separately, benchmark contamination undermines inference from performance on known tests to behavior on unseen traffic [50, 57].

2.3. Assumptions and Non-goals

Our conclusions rely on the following assumptions.

- A1. The reference endpoint is authentic and is not colluding with the tested vendor.
- A2. Reference and target requests use the same declared sampling configuration, and gross parameter rejection is checked before behavioral comparison.

- A3.** The reference supports next-token log probabilities and assistant-prefix continuation, as specified by the reference API [10, 11]. This capability is required only from the reference: without its conditional probabilities, the surprisal, entropy, and information variance are undefined. The target requires only ordinary text generation. A tail run is admitted only when the exact target prefix is accepted by the reference and all 500 prespecified positions receive valid conditional probabilities. This is an API-level replay-and-scoring gate, not evidence that the two routes use identical tokenizers.
- A4a.** The serving distribution is approximately stationary within one audit window. It may change across independently collected windows; this between-window variation is part of the process model rather than noise to discard.
- A4b.** Conditional on the declared request configuration and audit window, repeated requests within a context-route cell are treated as approximately independent draws from a fixed route distribution. Cache affinity, batching, or session routing can violate this approximation and produce overdispersion.
- A5.** The collected reference category vector is treated as fixed for inference. Sampling uncertainty in that API-returned vector and reference drift across serving windows are not propagated into the reported intervals or null calibration.
- A6.** A runtime nonce is secret before a challenge is sent, but the provider can inspect the challenge on arrival. Freshness therefore prevents exact precomputation, not semantic probe detection.

Ventor-QTest is not a proof of model ownership, remote attestation, or malicious intent. It does not identify a quantization level from a deviation score, certify downstream quality, or guarantee detection when a provider can perfectly distinguish audit traffic. The audit complements capability tests such as KVV: a deployment can be distributionally close on a short text-generation probe yet fail tool calling, multimodal preprocessing, long-context inference, or agentic execution.

3. Method

3.1. Overview

Ventor-QTest is a composite black-box audit with complementary AFL and EFL components. As shown in Figure 1, the repeated-request AFL component reconstructs finite conditional output distributions from repeated target text and returns a bias-corrected, within-window coarsened-KL statistic. The long-sequence EFL component repeats independently generated probes and retains the empirical distribution of all observed run-level deviations instead of collapsing it to an average or binary state. These observables have different units and are reported as a pair; they are never added into an arbitrarily weighted scalar. The procedure is specified in Section A.

3.2. Repeated-request Outcome Model

For each context x_j , a predeclared total map g_j assigns every possible returned string to a finite alphabet $\mathcal{A}_j$. The trusted reference induces category probabilities

$$\pi_{ji} = \Pr_p[g_j(Y) = i \mid x_j], \quad i \in \mathcal{A}_j. \quad (6)$$

The verifier then sends the exact same context to target route r a total of M times. Under the conditional-independence approximation in A4b, its category counts satisfy

$$(N_{jr1}, \dots, N_{jr|\mathcal{A}_j|}) \sim \text{Multinomial}(M, \theta_{jr}), \quad \theta_{jri} = \Pr_{Q_r}[g_j(Y) = i \mid x_j]. \quad (7)$$

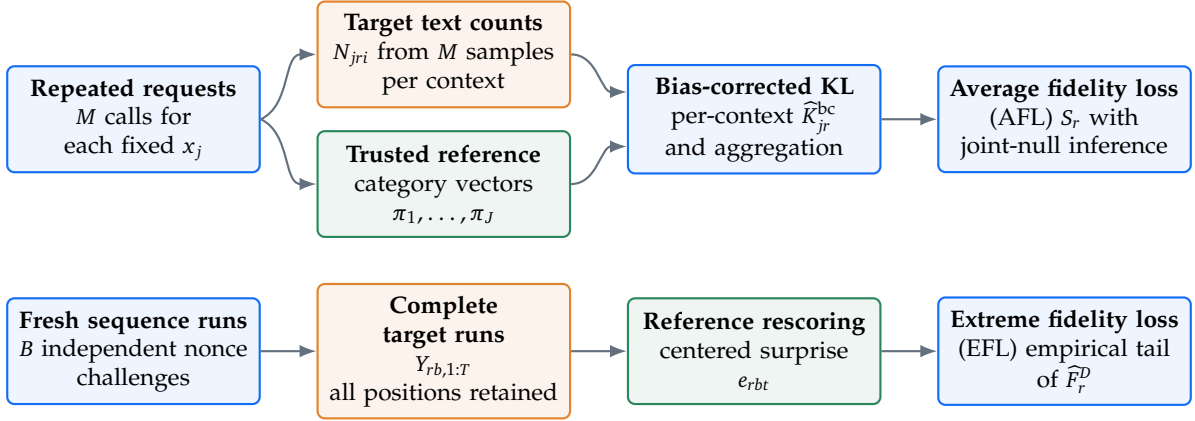


Figure 1 | Composite Ventr-QTest pipeline. Repeated requests to fixed contexts report AFL as a within-window mean coarsened-KL statistic, while independent long-sequence runs report EFL from the empirical upper tail of all observed centered-surprisal deviations. Both components require probabilities only from the trusted reference. Their outputs have different units and are reported jointly rather than summed into a scalar.

This sampling design identifies θ_{jr} from ordinary text. Whitespace-only, multi-token, missing, or otherwise nonconforming responses map to OTHER, so every request contributes to the likelihood.

Low-reference-mass categories are pooled before target sampling. Any declared category satisfying $M\pi_{ji} < c$ is merged into OTHER; the primary protocol fixes $c = 1$. If T_j denotes this deterministic coarsening, the data-processing inequality gives

$$D_{\text{KL}}(T_j Q_r \| T_j P) \leq D_{\text{KL}}(Q_r \| P). \quad (8)$$

The estimand is therefore a conservative task-coarsened divergence with a fully specified outcome map, by the data-processing inequality [8]. Because pooling reads only π_j , the target sample cannot influence category selection.

3.3. Finite-sample KL Estimator

A reference-centered Dirichlet posterior stabilizes multinomial KL estimation while preserving the reference distribution as the prior mean. We set

$$\theta_{jr} \sim \text{Dirichlet}(\tau \pi_j), \quad a_{jri} = N_{jri} + \tau \pi_{ji}, \quad A_0 = M + \tau, \quad (9)$$

with total prior strength $\tau = 1$. The posterior expected coarse KL is

$$\widehat{K}_{jr}^{\text{post}} = \sum_i \frac{a_{jri}}{A_0} [\psi(a_{jri} + 1) - \psi(A_0 + 1) - \log \pi_{ji}], \quad (10)$$

where ψ is the digamma function. The expression follows from the standard Dirichlet moment for $\mathbb{E}[\theta_i \log \theta_i]$ [52].

Finite-sample posterior uncertainty produces a positive baseline even under the null. We estimate and subtract that baseline separately for each context:

$$b_j(P, M) = \mathbb{E}_{N^{(0)} \sim \text{Multinomial}(M, \pi_j)} [\widehat{K}^{\text{post}}(N^{(0)}, \pi_j)], \quad (11)$$

$$\widehat{K}_{jr}^{\text{bc}} = \widehat{K}_{jr}^{\text{post}} - b_j(P, M), \quad S_r = \frac{1}{J} \sum_{j=1}^J \widehat{K}_{jr}^{\text{bc}}. \quad (12)$$

The implementation uses 20,000 parametric-null draws to estimate b_j . We refer to S_r as the route’s AFL in the frozen audit window. Negative finite-sample values are retained because clipping would bias correlations and route tests upward. A prior-free Pearson-divergence U-statistic, defined in Section B, checks whether the empirical ordering depends on the Dirichlet correction.

When a window index is needed, we write $S_{r,b}$ for the value of S_r computed separately in window b . For B independently spaced repeated-request windows, $B^{-1} \sum_b S_{r,b}$ would be a sample analogue of A_r under the stated reference-stability and sampling assumptions, where each $S_{r,b}$ is calculated before pooling observations across windows. The primary matched-route collection below has one window and therefore reports only its within-window context average; it does not directly estimate the cross-window quantity A_r .

3.4. Auxiliary Comparator and Route Inference

When a route exposes top log probabilities, its alternatives are aggregated through the same g_j to obtain a logprob-derived coarsened-KL comparator. This provider-controlled field is an auxiliary consistency check rather than ground truth and never enters the text-count statistic. Association is measured after centring both quantities within context. Exact significance permutes a common route-condition label assignment across all contexts, preserving the crossed repeated-measures structure rather than treating context–route cells as independent routes.

The implemented mean-component test concerns the finite probe null

$$H_{0,r}^{\text{mean}} : \theta_{jr} = \pi_j, \quad j = 1, \dots, J. \quad (13)$$

The statistic S_r is compared with a joint null generated by independent multinomial draws from the fixed π_j across all J contexts. One-sided p values use 20,000 draws, and the Holm procedure controls family-wise error across the matched routes [22]. Rejecting Equation (13) establishes inconsistency only for the frozen contexts, coarsened outcome maps, declared request configuration, and collection window. It does not directly reject the universal consistency condition in Equation (2). The request counts are

$$N_{\text{target}} = RJM, \quad N_{\text{reference}} = J, \quad (14)$$

because each reference vector is shared by all R routes.

3.5. Run-Level Tail Probe

The run-level component preserves deviations that a route average can hide. In independent run b , target route r first generates a length- T sequence $Y_{rb,1:T}$. At position t , the trusted reference supplies the conditional distribution $p_{rbt}(\cdot) = P(\cdot \mid Y_{rb,<t})$. Using the standard information-theoretic notions of surprise, entropy, and information variance [27, 43], we calculate the centered reference surprise

$$e_{rbt} = -\log p_{rbt}(Y_{rb,t}) - H(p_{rbt}), \quad H(p) = -\sum_y p(y) \log p(y), \quad (15)$$

and the complete-run deviation

$$D_{r,b}(T) = \left| \frac{1}{T} \sum_{t=1}^T e_{rbt} \right|. \quad (16)$$

If the target follows the reference conditional distribution, $e_{r|t}$ has zero conditional expectation, giving a martingale-difference structure under the reference null [18]. Because positions share an autoregressive history, however, the statistical unit is the complete sequence, not an individual token. All intervals therefore resample independent runs as clusters, following the nonparametric bootstrap principle [15].

Across B runs, the empirical distribution $\widehat{F}_r^D$ records both typical and upper-tail stochastic deviation. We refer to the observed upper-tail behavior of $\widehat{F}_r^D$ as the route’s EFL and report its median and standard deviation together with an upper empirical quantile and the observed maximum. These are descriptive summaries of the observed runs, not estimates of the latent $X_r(m)$ in Equation (4).

The sequence statistic is not a KL estimator. Under target distribution Q , its signed expectation contains $D_{\text{KL}}(Q\|P) + H(Q) - H(P)$, so an unknown entropy difference can offset KL. The composite audit therefore assigns distinct roles to its components and returns

$$\mathcal{V}_r = \left(S_r, \widehat{F}_r^D \right), \quad (17)$$

where S_r reports AFL over the frozen contexts and $\widehat{F}_r^D$ reports the empirical run-to-run distribution from which EFL is summarized. No numerical conversion between the two statistics, or from either statistic to task success, is assumed.

4. Evaluation

4.1. Evaluation Design

The experiments evaluate three questions. We first compare repeated-request AFL with an auxiliary logprob-derived comparator, then ask whether independent runs reveal EFL hidden by typical deviation, and finally examine their downstream relationships. The AFL, EFL, and downstream studies cover seven routes. Frozen protocol details and the configuration fields preserved by the collection artifacts appear in Appendix B, including Sections B.2 and B.3; fields not preserved are stated explicitly rather than reconstructed after the fact.

4.2. Descriptive Agreement with a Logprob-Derived Coarsened-KL Comparator

The text-count AFL statistic shows strong linear descriptive agreement with the comparator derived from log probabilities. After removing context fixed effects, Pearson correlation is $r = 0.971$ and Spearman correlation is $\rho = 0.657$. Among the $3! = 6$ exact common route permutations, both one-sided tests give $p = 0.167$; the route-limited result is therefore descriptive rather than confirmatory. Averaging first by route gives Pearson $r = 0.989$ and perfect rank agreement. The prior-free Pearson-divergence sensitivity and both stricter reference-only pooling thresholds retain positive association; complete values appear in Section B. Because the comparator is supplied by the target route, this analysis checks internal consistency rather than validating against independent ground truth.

4.3. Matched-route AFL Audit

The official control is centered at zero and is not rejected, while all six third-party routes reject the finite-probe joint null after multiplicity correction. These results establish inconsistency only on the 12 frozen contexts, their coarsened outcome maps, the declared request configuration,

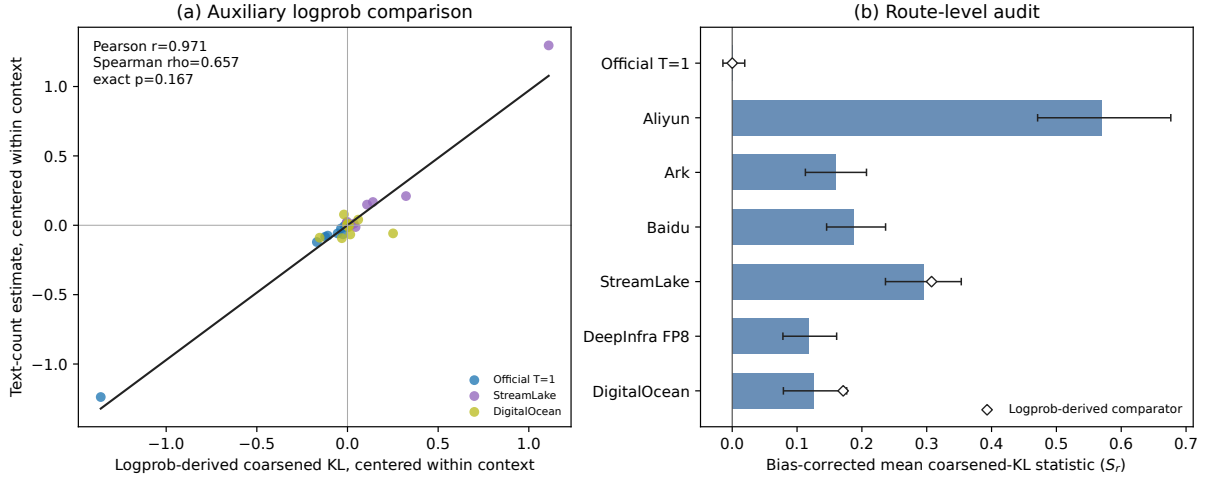


Figure 2 | Auxiliary consistency check for AFL. The left panel contains 36 context–route cells from three logprob-capable route conditions. Text-count AFL statistics and the logprob-derived coarsened-KL comparator show descriptive agreement after context centring; with only $3! = 6$ route permutations, the exact route-level test is not confirmatory. The right panel shows the seven-route audit, in which the official control remains at the route-level null.

Table 2 | AFL audit over seven routes. S_r is the mean bias-corrected coarsened-KL statistic over 12 held-out contexts; intervals are 95% posterior credible intervals after subtracting the null baseline. The final column reports Holm-adjusted p values.

Route	S_r [95% CrI]	Logprob comparator	Holm p
Official self-check	−0.0007 [−0.0144, 0.0193]	0.0001	0.515
Aliyun 0731	0.5704 [0.4710, 0.6765]	—	0.00035
Ark 0731	0.1591 [0.1129, 0.2071]	—	0.00035
Baidu Qianfan 0731	0.1875 [0.1456, 0.2365]	—	0.00035
StreamLake	0.2950 [0.2364, 0.3533]	0.3075	0.00035
DeepInfra FP8	0.1185 [0.0782, 0.1611]	—	0.00035
DigitalOcean	0.1250 [0.0789, 0.1766]	0.1712	0.00035

and the collection window. They neither reject the universal null in Equation (2) nor identify a changed checkpoint, numerical precision, or serving policy.

4.4. EFL Is Not Captured by AFL

The long-sequence probe reveals EFL that AFL does not show. Each entry in Table 3 summarizes 20 complete 500-position runs; no run or high-deviation observation is removed. The median characterizes typical stochastic deviation, while the standard deviation, empirical 0.95 quantile, and maximum summarize EFL in the observed runs.

DigitalOcean is the route snapshot with the largest median, run-level standard deviation, empirical 0.95 quantile, and maximum EFL summaries in this collection. It has the largest observed deviation in 5 of the 20 paired runs and ranks in the top two in 9 runs; these counts are descriptive. StreamLake also has pronounced EFL despite the smallest median, illustrating the point of the composite report: AFL and EFL need not order route snapshots in the same way.

Table 3 | EFL summaries across seven route snapshots. All quantities summarize $D_{r,b}(500)$ over 20 paired runs. Smaller values indicate less observed absolute centered-surprise drift under this probe; the statistic is not a KL estimator and does not provide a universal ordering of model fidelity. The empirical $q_{0.95}$ uses NumPy’s linear interpolation and, with 20 runs, is descriptive.

Route	Median D_{500}	SD	Empirical $q_{0.95}$	Maximum
Official self-check	0.00545	0.00835	0.02484	0.03498
Aliyun 0731	0.00754	0.00819	0.02190	0.02924
Ark 0731	0.00679	0.00881	0.02633	0.02901
Baidu Qianfan 0731	0.00612	0.00534	0.01693	0.01975
StreamLake	0.00409	0.01390	0.04817	0.04969
DeepInfra FP8	0.00517	0.00806	0.01996	0.03497
DigitalOcean	0.00876	0.01676	0.05414	0.05960

In an exploratory replication, four additional DigitalOcean repeated-request batches give

$$S_r \in \{0.12505, 0.13667, 0.12092, 0.14797\},$$

with mean 0.13265 and raw SD 0.01220. Four observations are insufficient to estimate a stable cross-window upper tail. We therefore use the 20-run sequence study to describe the observed stochastic upper-tail phenomenon and do not claim that the four coarse-KL batches identify the latent $X_r(m)$.

4.5. Exploratory Downstream Observations by Task Exposure

Across routes, AFL and EFL have little detectable association with GPQA-Diamond accuracy, and AFL alone does not order Terminal-Bench success. In contrast, DigitalOcean—the route with the most pronounced observed EFL—shows a decline in Terminal-Bench pass rate from 82.6% in the lowest-exposure quartile to 13.6% in the highest-exposure quartile. This concurrence may arise because correctness in long-horizon tasks is more sensitive to extreme fidelity loss: longer trajectories contain more model decisions and therefore more opportunities for an extreme deviation to affect the final outcome. The comparator-relative gaps are not monotonic across the intermediate quartiles, so this mechanism remains an exploratory interpretation rather than a monotonic exposure law. Complete analyses appear in Section C.

5. Related Work

Model equality and API auditing. Gao et al. formulate vendor verification as a two-sample model equality test and report distributional inconsistencies among commercial Llama endpoints [17]. RUT maps a target response into a randomized rank using reference samples and tests rank uniformity under user-like prompts [58]. Cai et al. formalize adversarial model substitution, evaluate text- and logprob-based auditors under production nondeterminism, and motivate hardware-backed verification [5]. KBF targets relay and reseller APIs with knowledge-boundary fingerprints and evaluates low-fraction mixed routing [16]. Ventor-QTest instead estimates a within-window task-coarsened conditional-KL statistic from repeated target text and reference probabilities, requiring neither target logits nor a local target-weight replica.

API change tracking. Log Probability Tracking monitors API changes with one output token and target-provided log probabilities [7]. B3IT uses strict black-box “border inputs” and observes

only output tokens [6]; You’ve Changed instead compares distributions of linguistic features in generated text [14]. Ventr-QTest reconstructs a predeclared categorical distribution from repeated text and pairs its mean coarsened-KL statistic with an independent long-sequence empirical-tail measurement.

Behavioral fingerprints. LLMmap identifies versions with optimized queries [39]. FLIPS uses pseudo-random behavior to distinguish served configurations [42], while ESF optimizes sensitive token positions for black-box tamper detection [2]. These methods primarily address attribution, instance identity, or tamper detection. Ventr-QTest reports a reference-relative effect on a declared finite outcome space without training a classifier.

Concurrent work. Two July 2026 preprints are concurrent with this work. One Token Is Enough fingerprints and verifies models from empirical single-token output distributions [4]. IRIS audits whole-stream substitution and fractional routing dilution from text alone while sizing its own query budget [56]. Both reinforce the relevance of text-only distributional probes; Ventr-QTest differs by reporting separate within-window AFL and empirical run-level EFL rather than model attribution or routing-fraction estimation.

Capability verification. Capability benchmarks complement identity tests by evaluating task behavior, including OCR [28], multimodal reasoning [53], software engineering [26], mathematics [31], and broad knowledge [49, 50]. Kimi Vendor Verifier (KVV) applies this view to hosted deployments: its evaluations cover tool triggering and schema correctness, multimodal processing, long-output reasoning, and agentic coding [34, 36]. KVV therefore establishes the practical importance of deployment correctness and already recognizes benchmark fluctuation. Ventr-QTest formalizes a complementary dimension: it treats a route as a distribution over serving windows and separates persistent mean deviation from intermittent upper-tail deviation before a full capability benchmark is run.

Query budget and monetary cost. As Table 4 shows, request counts purchase different forms of evidence. Gao et al.’s 200–250 target calls denote one two-sample test, with an equal reference sample; their endpoint verdict aggregates 10 repeated tests. LLMmap’s setup is per reference model: 75 configurations times eight queries for training and a disjoint construction of the same size for testing, distinct from its 3–8-call online verification. Fingerprinting methods minimize online verification cost by amortizing a trained reference classifier, RUT shifts most inference to a local reference, and KVV spends substantially more target tokens to measure functional capability directly. Ventr-QTest separates a target-side inexpensive mean component from a reference-intensive run-level component. The mean audit uses one-token target calls, shares 12 reference vectors across routes, and returns a finite-probe coarsened-KL statistic with a calibrated decision at a declared power target (Sections D and E); the run-level audit spends 10,000 reference prefix-rescoring calls per route to preserve 20 complete observations. Monetary accounting is reported only for the mean component whose exact metering is available.

Adaptive audit recognition. Public probes create a selective-routing threat. LLMmap analyzes query-informed perturbation of recognized fingerprint traffic, RUT uses ordinary task prompts to reduce recognizability, KBF and IRIS test mixed or diluted routing, and GhostPrint demonstrates finite-budget fingerprint imitation [16, 39, 55, 56, 58]. Ventr-QTest’s contexts

Table 4 | Representative operating points for black-box verification methods. Counts are not matched-power comparisons. Vantor-QTest’s mean component is target-side inexpensive, whereas its run-level component is reference-intensive.

Method	Target calls	Reference/setup	Audit objective and limitation
Gao et al. [17]	200–250 per single test; 10 tests/verdict	Matched 200–250 reference samples per test	Distributional equality/distance; adaptive routing not explicit
RUT [58]	100	10,000 local generations and logits	Rank uniformity on user-like prompts; probabilistic substitution model
LLMmap [39]	3–8 online	About 1,200 sourcing calls per reference model over train/test; trained classifier	Closed-set version identity; analyzes query-informed defenses
FLIPS [42]	8	40 extraction calls; trained classifier	Closed/open-set instance identity; assumes a non-adaptive target
KVV [34, 36]	4,000	Official responses and labels	Functional behavior; a finite suite can be recognized
Vantor-QTest	Mean: 600 one-token; run level: 20 sequences	Mean: 12 shared probability calls; run level: 10,000 prefix-rescoring calls/route	Finite-probe mean statistic plus empirical run tail; no guarantee against perfect recognition

are therefore evidence only for the stated audit distribution; prompt secrecy is not treated as protection against a provider that recognizes the task family.

6. Discussion and Limitations

Measurement scope. The mean component reports coarsened KL only on the declared maps and within the observed window. The run-level component measures centered-surprise deviation rather than KL. The present sample supports comparison of observed empirical distributions but not precise rare-event probabilities or estimation of $X_r(m)$. Multinomial calibration treats within-cell requests as approximately independent and the returned reference vectors as fixed; correlated routing, overdispersion, reference uncertainty, or reference drift can make intervals and null p values optimistic. Like any behavioral audit, Vantor-QTest cannot detect deviations outside its probe distribution or defeat a provider that perfectly recognizes audit traffic.

Downstream scope. The paper does not posit a monotonic mapping from AFL or EFL to task success. A deviation may hurt, leave unchanged, or improve a particular task, and benchmark outcomes also reflect timeouts, rate limits, tool semantics, and agent-runtime behavior. The observed concurrence between pronounced EFL and decreasing Terminal-Bench pass rate suggests that extreme fidelity loss may influence correctness in long-horizon tasks, but the unsynchronized probe and benchmark windows cannot distinguish this explanation from those alternatives. Confirming the causal contribution requires more independent windows and time-aligned probe–benchmark collection.

7. Conclusion

Ventor-QTest returns two observables that are not interchangeable: AFL, a null-bias-corrected within-window coarsened-KL statistic, and EFL, the empirical upper-tail behavior of run-level centered-surprisal drift. Experiments show descriptive agreement between AFL and a logprob-derived comparator, and show that AFL and EFL can order route snapshots differently.

The downstream comparison finds little association between either audit output and GPQA-Diamond accuracy, while pronounced EFL coincides with declining Terminal-Bench pass rate as task exposure grows. This pattern suggests that extreme fidelity loss may matter more for correctness in long-horizon tasks. The central methodological implication is to report AFL and EFL jointly while stating the finite probe scope of each inference.

References

- [1] Anthropic. Claude code: Overview. Official product documentation, 2026. URL <https://docs.anthropic.com/en/docs/claude-code/overview>. Accessed 2026-08-12.
- [2] X. Bai, P. Hu, X. Ma, L. Yu, D. Zhang, Q. Zhang, and B. B. Zhu. ESF: Efficient sensitive fingerprinting for black-box tamper detection of large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 10477–10494. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-acl.546. URL <https://aclanthology.org/2025.findings-acl.546/>.
- [3] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan. Remote ATtestation procedureS (RATS) architecture. RFC 9334, Internet Engineering Task Force, 2023. URL <https://www.rfc-editor.org/rfc/rfc9334>.
- [4] T. Bruckner. One token is enough: Fingerprinting and verifying large language models from single-token output distributions, 2026. URL <https://arxiv.org/abs/2607.10252>. Concurrent work submitted 2026-07-11.
- [5] W. Cai, T. Shi, X. Zhao, and D. Song. Are you getting what you pay for? auditing model substitution in LLM APIs, 2025. URL <https://arxiv.org/abs/2504.04715>.
- [6] T. Chauvin, C. Lalanne, E. Le Merrer, J.-M. Loubes, F. Taïani, and G. Tredan. Token-efficient change detection in LLM APIs. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2602.11083>.
- [7] T. Chauvin, E. Le Merrer, F. Taïani, and G. Tredan. Log probability tracking of LLM APIs. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2512.03816>.
- [8] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2 edition, 2006. doi: 10.1002/047174882X.
- [9] DeepSeek AI. Change log: DeepSeek-V4-Flash update. Official API documentation, July 2026. URL <https://api-docs.deepseek.com/updates/>. Version entry dated 2026-07-31; accessed 2026-08-17.
- [10] DeepSeek AI. Chat completions API. Official API reference, 2026. URL <https://api-docs.deepseek.com/api/create-chat-completion>. Accessed 2026-08-07.

- [11] DeepSeek AI. Chat prefix completion. Official API documentation, 2026. URL https://api-docs.deepseek.com/guides/chat_prefix_completion/. Accessed 2026-08-07.
- [12] DeepSeek AI. Models and pricing. Official API documentation, 2026. URL https://api-docs.deepseek.com/quick_start/pricing/. Dynamic page accessed 2026-08-14; the historical rates used in this paper are preserved in the released collection artifact.
- [13] DeepSeek AI. DeepSeek V4 preview release. Official API documentation, 2026. URL <https://api-docs.deepseek.com/news/news260424/>. Accessed 2026-08-07.
- [14] A. Dima, J. Foulds, S. Pan, and P. Feldman. You’ve changed: Detecting modification of black-box large language models, 2025. URL <https://arxiv.org/abs/2504.12335>.
- [15] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993. doi: 10.1201/9780429246593.
- [16] Y. Fang, Y. Feng, B. Li, and M. Zhou. KBF: Knowledge boundary as fingerprint for language model and black-box API auditing, 2026. URL <https://arxiv.org/abs/2605.29524>.
- [17] I. Gao, P. Liang, and C. Guestrin. Model equality testing: Which model is this API serving? In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=QCDDI7X3f9>.
- [18] P. Hall and C. C. Heyde. *Martingale Limit Theory and Its Application*. Academic Press, New York, 1980. ISBN 978-0-12-319350-6.
- [19] Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments. Software release, 2026. URL <https://github.com/harbor-framework/harbor/releases/tag/v0.21.0>. Version 0.21.0; includes the Terminus-2 agent; released 2026-08-10.
- [20] C. R. Harris, K. J. Millman, S. J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.
- [21] W. Hoeffding. A class of statistics with asymptotically normal distribution. *The Annals of Mathematical Statistics*, 19(3):293–325, 1948. doi: 10.1214/aoms/1177730196.
- [22] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL <https://www.jstor.org/stable/4615733>.
- [23] Hugging Face. State of open source on Hugging Face: Spring 2026. Hugging Face Blog, Mar. 2026. URL <https://huggingface.co/blog/huggingface/state-of-os-hf-spring-2026>. Accessed 2026-08-12.
- [24] E. J. Husom, A. Goknil, M. Astekin, L. K. Shar, A. Kåsen, S. Sen, B. A. Mithassel, and A. Soylu. Sustainable LLM inference for edge AI: Evaluating quantized LLMs for energy efficiency, output accuracy, and inference latency. *arXiv preprint arXiv:2504.03360*, 2025. URL <https://arxiv.org/abs/2504.03360>.
- [25] R. J. Hyndman and Y. Fan. Sample quantiles in statistical packages. *The American Statistician*, 50(4):361–365, 1996. doi: 10.1080/00031305.1996.10473566.

- [26] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- [27] I. Kontoyiannis and S. Verdú. Optimal lossless data compression: Non-asymptotics and asymptotics. *IEEE Transactions on Information Theory*, 60(2):777–795, 2014. doi: 10.1109/TIT.2013.2291007.
- [28] Y. Liu, Z. Li, M. Huang, B. Yang, W. Yu, C. Li, X.-C. Yin, C.-L. Liu, L. Jin, and X. Bai. OCRBench: On the hidden mystery of OCR in large multimodal models. *Science China Information Sciences*, 67(12):220102, 2024. doi: 10.1007/s11432-024-4235-6. URL <https://arxiv.org/abs/2305.07895>.
- [29] S. Longpre, C. Akiki, C. Lund, A. Kulkarni, E. Chen, I. Solaiman, A. Ghosh, Y. Jernite, and L.-A. Kaffee. Economies of open intelligence: Tracing power & participation in the model ecosystem. *arXiv preprint arXiv:2512.03073*, 2025. URL <https://arxiv.org/abs/2512.03073>.
- [30] H. B. Mann and D. R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947. doi: 10.1214/aoms/1177730491.
- [31] Mathematical Association of America. Invitational competitions: American invitational mathematics examination. Official competition documentation, 2026. URL <https://maa.org/maa-invitational-competitions/>. Accessed 2026-08-07.
- [32] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- [33] P. Micikevicius, D. Stolic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, and H. Wu. FP8 formats for deep learning, 2022. URL <https://arxiv.org/abs/2209.05433>.
- [34] Moonshot AI. K2 Vendor Verifier: Verify precision of all Kimi K2 API vendors. GitHub repository, 2025. URL <https://github.com/MoonshotAI/K2-Vendor-Verifier>. Accessed 2026-08-17.
- [35] Moonshot AI. Kimi K2 deployment guidance. Official deployment documentation, 2025. URL https://github.com/MoonshotAI/Kimi-K2/blob/main/docs/deploy_guidance.md. Accessed 2026-08-12.
- [36] Moonshot AI. Rebuilding the “chain of trust”: Kimi Vendor Verifier. Kimi Research Blog, 2026. URL <https://www.kimi.com/blog/kimi-vendor-verifier>. Accessed 2026-08-07.
- [37] OpenAI. Codex: AI coding agents for software engineering. Official product documentation, 2026. URL <https://openai.com/codex/>. Accessed 2026-08-12.
- [38] OpenRouter. Provider routing. Official documentation, 2026. URL <https://openrouter.ai/docs/guides/routing/provider-selection>. Accessed 2026-08-07.

- [39] D. Pasquini, E. M. Kornaropoulos, and G. Ateniese. LLMmap: Fingerprinting for large language models. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 299–318. USENIX Association, 2025. URL <https://www.usenix.org/conference/usenixsecurity25/presentation/pasquini>.
- [40] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- [41] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA dataset, diamond split. Hugging Face dataset, 2026. URL <https://huggingface.co/datasets/idavidrein/gpqa>. Accessed 2026-08-12.
- [42] G. Richardeau, G. Dashyan, E. Le Merrer, and G. Tredan. FLIPS: Instance-fingerprinting for LLMs via pseudo-random sequences. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2606.03330>.
- [43] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27 (3–4):379–423, 623–656, 1948. doi: 10.1002/j.1538-7305.1948.tb00917.x.
- [44] SiliconFlow. Create chat completion (OpenAI). Official API reference, 2026. URL <https://docs.siliconflow.cn/cn/api-reference/chat-completions/chat-completions>. Accessed 2026-08-12.
- [45] Terminal-Bench Team. Terminal-Bench 2.1 dataset. Official dataset repository, 2026. URL <https://github.com/harbor-framework/terminal-bench-2-1>. Version 2.1; 89 tasks; accessed 2026-08-17.
- [46] A. W. van der Vaart. *Asymptotic Statistics*. Cambridge University Press, 1998. doi: 10.1017/CBO9780511802256.
- [47] P. Virtanen, R. Gommers, T. E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [48] R. Wang, Y. Gong, X. Liu, G. Zhao, Z. Yang, B. Guo, Z. Zha, and P. Cheng. Optimizing large language model training using FP4 quantization. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 62937–62957, 2025. URL <https://arxiv.org/abs/2501.17116>.
- [49] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL <https://arxiv.org/abs/2406.01574>.
- [50] C. White, S. Dooley, M. Roberts, A. Pal, B. Feuer, S. Jain, R. Schwartz-Ziv, N. Jain, K. Saifullah, S. Dey, Shubh-Agrawal, S. S. Sandha, S. V. Naidu, C. Hegde, Y. LeCun, T. Goldstein, W. Neiswanger, and M. Goldblum. LiveBench: A challenging, contamination-limited LLM benchmark. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2406.19314>.

- [51] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- [52] D. H. Wolpert and D. R. Wolf. Estimating functions of probability distributions from a finite set of samples. *Physical Review E*, 52(6):6841–6854, 1995. doi: 10.1103/PhysRevE.52.6841.
- [53] X. Yue, T. Zheng, Y. Ni, Y. Wang, K. Zhang, S. Tong, Y. Sun, B. Yu, G. Zhang, H. Sun, Y. Su, W. Chen, and G. Neubig. MMMU-pro: A more robust multi-discipline multimodal understanding benchmark. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15134–15186. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.736. URL <https://aclanthology.org/2025.acl-long.736/>.
- [54] C. Zeng, S. Liu, Y. Xie, H. Liu, X. Wang, M. Wei, S. Yang, F. Chen, and X. Mei. ABQ-LLM: Arbitrary-bit quantized inference acceleration for large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 22299–22307, 2025. doi: 10.1609/aaai.v39i21.34385. URL <https://ojs.aaai.org/index.php/AAAI/article/view/34385>.
- [55] J. Zhang, X. Li, and S. Wang. Your “pro” LLM subscription may actually be “free”: Exposing fingerprint spoofing risks in LLM inference services, 2026. URL <https://arxiv.org/abs/2606.16100>.
- [56] Y. Zhang, Z.-H. Zhang, and H. Qin. Which model is actually serving you? IRIS: Budgeted black-box auditing of model substitution and routing dilution in LLM gateways, 2026. URL <https://arxiv.org/abs/2607.20860>. Concurrent work submitted 2026-07-23.
- [57] Q. Zhu, Q. Cheng, R. Peng, X. Li, R. Peng, T. Liu, X. Qiu, and X. Huang. Inference-time decontamination: Reusing leaked benchmarks for large language model evaluation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 9113–9129. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-emnlp.532. URL <https://aclanthology.org/2024.findings-emnlp.532/>.
- [58] X. Zhu, Y. Ye, T. Qiu, H. Zhu, S. Tan, A. Mannan, J. Michala, R. A. Popa, and W. Neiswanger. Auditing black-box LLM APIs with a rank-based uniformity test. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=PSIe9m mF7a>.

A. Audit Procedure

For fixed contexts $x_1, \dots, x_J$, target routes $Q_1, \dots, Q_R$, and trusted reference P , the implemented audit executes the following steps:

1. Freeze every context, total outcome map g_j , repetition count M , pooling threshold c , prior strength τ , multiplicity family, and analysis seed before target collection.
2. Query the reference once per context, aggregate exposed probabilities through g_j , and place all residual probability mass in OTHER.
3. Merge any declared category satisfying $M\pi_{ji} < c$ into OTHER; this map is fixed using the reference alone.
4. Query each target route M times with the exact same context and declared sampling settings.

5. Map every returned string to one category. Nonconforming text remains in OTHER; no target response is deleted.
6. Compute Equations (10) and (12); save raw text, counts, reference categories, route statistics, null calibration, and request metadata.
7. Simulate the joint route null, calculate one-sided route p values, and apply Holm correction over the frozen matched-route family [22].
8. If a route exposes top log probabilities, store them separately and compute a logprob-derived coarsened-KL comparator only as an auxiliary consistency check.
9. Independently of the mean audit, collect complete long-sequence runs, rescore every retained target position with the trusted reference, and report the empirical run-level distribution without deleting extreme observations.

A.1. Outcome Mapping and Reference Support

Each one-token task declares two to four exact digit labels. An exact returned label enters its named category; whitespace-only, multi-token, missing, or otherwise nonconforming text enters OTHER. The same map is applied to reference probability mass, target text counts, and the optional target-logprob comparator. Consequently every target request contributes to the multinomial count.

The reference request exposes the top 20 token probabilities. Probabilities of declared labels are retained when present, and all remaining probability is assigned to OTHER; the augmented vector is renormalized. Reference-only pooling then merges categories whose expected count is below c . Target output never changes this support.

B. Experimental Protocol

The protocol was frozen before formal collection. It uses the requested DeepSeek V4 Flash 0731 revision, $J = 12$ fixed contexts disjoint from five method-development contexts, $M = 50$ target samples per context-route cell, matched temperature 1, a one-token output limit, $\tau = 1$, $c = 1$, 20,000 parametric-null draws, and analysis seed 20260814. The official reference returns its top 20 log probabilities. Pooling thresholds $c \in \{2, 5\}$ are prespecified sensitivities. The official V4 documentation establishes the model family [13]. Here “0731” denotes the requested revision label, not the collection date: the 2026-07-31 change log names DeepSeek-V4-Flash-0731 and describes it as retaining the Preview architecture and size with new post-training [9].

The formal audit contains seven matched routes, each evaluated on the same 12 contexts and reference vectors with $M = 50$, temperature 1, and a one-token limit. Target log probabilities are not requested from Aliyun, Ark, Baidu, or DeepInfra.

The contexts are fair or weighted virtual choices over prompt-declared digit supports. Exact prompt strings and supports are present in the released protocol and summary artifacts. The formal matched design contains 4,200 one-token target calls and 12 shared reference calls. The normalized artifacts do not preserve provider region, top_p, stream mode, timeout, connection reuse, conversation or session identifiers, per-request seed support, randomized route and context interleaving, or retry lineage. We therefore do not claim those controls were randomized or standardized, and route-time correlation, correlated routing, and unreconstructable retries remain threats to calibration and exact operational reproduction.

Table 5 | Configuration fields preserved for the repeated-request audit. The requested revision is DeepSeek V4 Flash 0731 for every row. Optional target log probabilities are provider-controlled and never enter the text-count statistic.

Route snapshot	Access path	Precision status	Target logprobs
Official self-check	Official API	not declared	yes
Aliyun 0731	Aliyun Model Studio	not declared	no
Ark 0731	Volcengine Ark	not declared	no
Baidu Qianfan 0731	Baidu Qianfan	not declared	no
StreamLake	OpenRouter	not declared	yes
DeepInfra FP8	OpenRouter	provider-declared FP8	no
DigitalOcean	OpenRouter	not declared	yes

B.1. Auxiliary Comparator and Sensitivity Inference

The auxiliary comparison centers $\hat{K}_{jr}^{bc}$ and the logprob-derived comparator within each context. It reports Pearson and Spearman association across the complete route–context grid with target log probabilities. The exact one-sided test enumerates a common permutation of route-condition labels across all contexts, preserving each route condition as a repeated-measures block. Route-level detection averages the 12 context estimates, simulates their joint null under independent multinomials from the fixed reference probabilities, and applies Holm correction across the seven matched routes.

The prior-free sensitivity statistic is the following order-two unbiased Pearson-divergence U-statistic [21]:

$$\hat{D}_{\chi^2, jr} = \sum_i \frac{N_{jri}(N_{jri} - 1)}{M(M - 1)\pi_{ji}} - 1, \quad \mathbb{E}[\hat{D}_{\chi^2, jr}] = D_{\chi^2}(\theta_{jr} \| \pi_j). \quad (18)$$

Expanding $u \log u$ around $u = 1$ gives, for local alternatives, $D_{KL} = \frac{1}{2}D_{\chi^2} + O(\|\theta/\pi - 1\|^3)$, so $\hat{D}_{\chi^2}/2$ checks estimator dependence without a Bayesian prior.

Averaging the primary estimates first by route gives Pearson $r = 0.989$ and perfect rank agreement with the logprob-derived comparator. The prior-free statistic has context-centered Pearson $r = 0.926$ and Spearman $\rho = 0.560$. Raising the minimum reference expected count from $c = 1$ to $c = 2$ and $c = 5$ changes centered Pearson from 0.971 to 0.942 and 0.951; the corresponding centered Spearman values are 0.657, 0.653, and 0.755. With three matched, logprob-capable route conditions, every exact one-sided common-route permutation test gives $p = 1/6 = 0.167$ at the observed ordering. This is descriptive agreement with a target-controlled field, not confirmatory validation against ground truth.

Four additional DigitalOcean batches were collected after the primary matched-route window. They are labeled exploratory replications rather than part of the main route comparison. They show the observed dispersion of $S_{r,b}$ over four additional windows; four batches are insufficient to estimate a stable cross-window mean A_r or the latent excess maximum $X_r(m)$.

B.2. Long-Sequence Run-Level Protocol

The run-level study uses 20 paired replicates for each of Official, Aliyun, Ark, Baidu Qianfan, StreamLake, DeepInfra FP8, and DigitalOcean. Within a replicate, all seven routes receive the same freshly generated nonce challenge; across replicates, nonces are independent. Each admitted target sequence contains all 500 prespecified scored positions. A separate set of 20

official-control sequences uses disjoint nonces, producing 160 complete sequences and 80,000 scored positions in total. The Official row in Table 3 uses the 20 paired Official runs. The disjoint Official controls are retained as a diagnostic partition in the released run-level data and do not enter the reported route table, figures, or route comparisons.

For every target position, the trusted reference returns the conditional probability distribution under the exact target prefix. The analysis stores observed surprise, reference entropy, information variance, and the centered residual in Equation (15). It computes $D_{r,b}(n)$ at the prespecified position budgets

$$n \in \{10, 20, 30, 40, 50, 75, 100, 125, 150, 175, 200, 250, 300, 350, 400, 450, 500\}.$$

The complete sequence is the statistical unit. Pointwise 95% intervals resample the 20 runs as clusters with 20,000 bootstrap draws; the same sampled run indices are reused across all n to preserve paired convergence differences. No control-derived cutoff or binary stability label is constructed.

The primary run-level report uses all 20 values of $D_{r,b}(500)$. It reports the median, standard deviation, empirical quantiles, and maximum. The empirical $q_{0.95}$ uses NumPy 1.26.4’s default `method=linear`, corresponding to the commonly used type-7 sample quantile [20, 25]. The maximum is an observed sample statistic rather than an estimate of the population endpoint, and none of these summaries estimates $X_r(m)$.

The released analysis environment is recorded in `requirements-analysis.txt`: Python 3.11.6, NumPy 1.26.4, SciPy 1.17.1, and Matplotlib 3.11.1. SciPy supplies the reported correlations, Mann–Whitney test, and central and noncentral chi-square functions [47]; bootstrap resampling is implemented directly with NumPy.

B.3. Downstream Evaluation Protocol

GPQA-Diamond uses all 198 items in the benchmark’s highest-quality subset [40, 41] for each of the seven routes in Table 3. The frozen item order and answer parser are shared across routes. Reported route accuracies use Wilson 95% intervals [51]; Pearson and Spearman significance enumerates all $7!$ route-label permutations.

Terminal-Bench 2.1 uses all 89 tasks per route. We cite both the Terminal-Bench paper, which introduces the 89-task 2.0 benchmark, and the official 2.1 dataset used here [32, 45]. Runs use the Terminus-2 agent distributed with Harbor 0.21.0 [19]. The benchmark was collected from 2026-08-13 10:09 to 2026-08-15 02:51 China Standard Time and is an exploratory observational study, not a time-aligned continuation of the audit windows. Confirmed provider-side `APIError` trials for Ark and Qianfan are replaced only by targeted same-task reruns. Final `AgentTimeoutError`, `VerifierTimeoutError`, and runtime-error outcomes remain zero-reward observations. DigitalOcean was collected as a separate replication under the same task protocol. The task-exposure proxy is the median model-request count for the same task across the five non-DigitalOcean matched routes and uses no DigitalOcean-derived complexity measure. Quartile uncertainty resamples whole tasks with 20,000 bootstrap draws [15].

C. Downstream Benchmark Results

All seven routes answer every GPQA-Diamond item, and all scored responses parse successfully. Accuracy spans 69.7–74.7% with overlapping Wilson intervals. Median D_{500} has Pearson $r =$

0.347 (exact $p = 0.446$) and Spearman $\rho = 0.464$ ($p = 0.302$) with accuracy, providing no evidence of a negative route-level association.

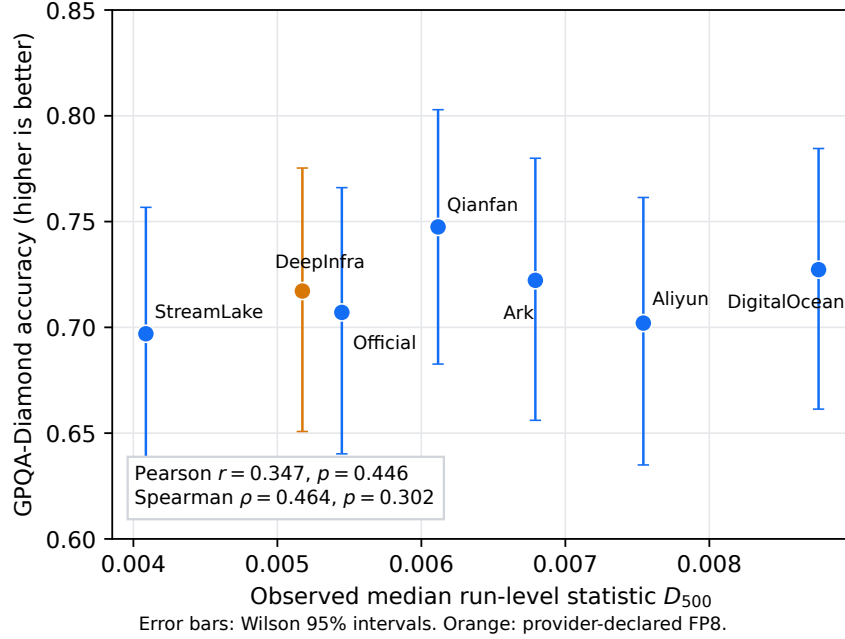


Figure 3 | GPQA-Diamond accuracy versus median run-level D_{500} . Error bars are Wilson 95% intervals; the route-level association is not statistically supported.

Terminal-Bench produces one terminal outcome for every task and route. Only confirmed provider-side `APIError` trials are replaced by targeted same-task reruns: 57 for Ark and 14 for Qianfan. All final agent, verifier, and runtime failures remain zero-reward outcomes. DigitalOcean is collected as a separate replication under the same task protocol.

Table 6 | Complete Terminal-Bench 2.1 route results after targeted API-error replacement. Every denominator is 89 tasks.

Route	S_r	Passes	Pass rate $\uparrow$	Errors $\downarrow$
Official	-0.0007	48	53.9%	22
Aliyun 0731	0.5704	45	50.6%	26
Ark 0731	0.1591	53	59.6%	14
Baidu Qianfan 0731	0.1875	38	42.7%	32
StreamLake	0.2950	52	58.4%	22
DeepInfra FP8	0.1185	46	51.7%	23
DigitalOcean	0.1250	33	37.1%	48

Across the seven routes, S_r has Pearson $r = 0.081$ (exact $p = 0.873$) and Spearman $\rho = -0.036$ ($p = 0.963$) with Terminal-Bench pass rate. The within-window mean statistic alone therefore does not order Terminal-Bench performance.

For the exposure analysis, tasks are sorted by the median model-request count for the same task across five matched non-DigitalOcean routes. This construction contains no DigitalOcean-derived complexity measure. All 47 DigitalOcean `AgentTimeoutError` outcomes remain failures because removing them would condition on the observed outcome.

Table 7 | DigitalOcean success across task-exposure quartiles. The comparator is the same-task mean over the five matched routes used to construct exposure.

Exposure quartile	Tasks	DO pass rate	Comparator pass rate	DO agent timeouts
Q1 (shortest)	23	82.6%	76.5%	4
Q2	22	31.8%	62.7%	10
Q3	22	18.2%	37.3%	16
Q4 (longest)	22	13.6%	28.2%	17

DigitalOcean’s comparator-relative gaps are +6.1, −30.9, −19.1, and −14.6 percentage points from Q1 to Q4, so the deficit is not monotonic across quartiles. The prespecified endpoint contrast shows that the Q4 gap is 20.6 percentage points worse than the Q1 gap; the task-bootstrap 95% interval is $[-36.0, -4.5]$ percentage points and the one-sided nonnegative-tail probability is 0.0059. Failed DigitalOcean tasks also have larger exposure than successful tasks (median 36.5 versus 11 requests; one-sided Mann–Whitney $p = 3.87 \times 10^{-7}$) [30]. However, 47 of its 56 non-pass outcomes (84%) are `AgentTimeoutError` outcomes. The serving deviations reflected by Ventr-QTest’s empirical upper-tail statistic may have contributed to these failures, but the lack of time-aligned probe and benchmark windows prevents confirmed causal attribution.

D. Fixed-sample Request Planning

A finite request budget is defined relative to a minimum effect of interest. For the frozen contexts, a route’s within-window mean coarsened KL is

$$\delta = J^{-1} \sum_j D_{\text{KL}}(\theta_{jr} \| \pi_j). \quad (19)$$

For acquisition planning, the local asymptotic multinomial likelihood-ratio model [46] gives

$$\nu(M) = \sum_{j=1}^J \{k_j(M) - 1\}, \quad G_r^2 \sim \chi_{\nu(M)}^2(\lambda), \quad \lambda \simeq 2JM\delta, \quad (20)$$

where $k_j(M)$ is the number of categories remaining after reference-only pooling. For R simultaneous route comparisons, planning uses the conservative threshold $\alpha_* = 0.05/R$. Approximate power is

$$\mathcal{P}(M, \delta) = 1 - F_{\chi_{\nu(M)}^2(2JM\delta)}(\chi_{\nu(M), 1-\alpha_*}^2). \quad (21)$$

Integer M is enumerated after applying the same $M\pi_{ji} \geq 1$ pooling rule used by the estimator, so support changes are included in $\nu(M)$.

At $M = 50$, $\nu = 24$, and inversion of Equation (21) gives a 90% planning effect floor of $\delta = 0.0308$. The weakest observed third-party score is 0.1185, although the comparison is descriptive because the effect is estimated from the audit data. Equations (20)–(21) are local-asymptotic planning approximations; actual route claims use the 20,000-draw protocol-specific null.

E. Mean-Component Cost Accounting

The following calculation covers only the repeated-request mean component. The 12 exact contexts were metered once against official DeepSeek V4 Flash with one output token per

Table 8 | Fixed-sample planning for $J = 12$, 90% power, and family-wise level 0.05 across $R = 6$ route comparisons. Calls are one-token target requests per route; 12 reference distributions are shared across routes.

Minimum mean coarsened KL, δ	Repeats/context, M	Target calls/route	$\nu(M)$
0.010	159	1,908	26
0.020	79	948	25
0.030	52	624	24
0.050	30	360	21
0.100	15	180	19

request. One complete pass uses 432 input and 12 output tokens. At the rates captured on 2026-08-14 in the released artifact `data/repeated_context_cost.json`, cache-miss input cost \$0.14 and output cost \$0.28 per million tokens. The artifact records the then-current official pricing page [12]; because that page is mutable, the artifact rather than its present contents is the evidence for these historical rates. All input is conservatively charged as a cache miss. A 600-call target route uses 21,600 input and 600 output tokens; the shared reference pass adds 432 input and 12 output tokens. The single-route list-price equivalent is

$$C_1 = \frac{(21,600 + 432)(0.14) + (600 + 12)(0.28)}{10^6} = \$0.00325584 \approx \$0.0033. \quad (22)$$

The seven-route matched audit costs \$0.0224 under these assumptions. Cache-hit discounts, provider-specific pricing, retries, taxes, and minimum charges are excluded.

F. Released Experimental Data

The release includes the frozen protocols, categorized counts for 4,200 matched-route responses, 12 reference category distributions, context-level estimates, route-level joint-null tests, and pooling sensitivities. It also includes all 20 complete D_{500} values for each of seven paired routes, convergence summaries, complete 198-item GPQA results, all 89 Terminal-Bench task outcomes per route, and the task-exposure audit. The JSON and CSV artifacts reproduce Figures 2 and 3 and Tables 2, 3 and 6 to 8; authorization headers and API keys are excluded.